Skills and Subskills

Skill 1 — Git

Preparedness target: STRONG

Git is relatively easy compared with the rest, so we don't need 30 subskills here. But you should be able to use Git comfortably in a real engineering environment and answer common interview questions.

Subskills

1. Git repository fundamentals
    - repository
    - working tree
    - staging area
    - commits
    - HEAD
    - branches
2. Basic Git workflow
    - `clone`
    - `status`
    - `add`
    - `commit`
    - `push`
    - `pull`
    - `fetch`
3. Branching and merging
    - branch creation

- checkout/switch
- merge
- fast-forward merge
- merge commits
- branch deletion

4. Merge conflicts
   - identifying conflicts
   - resolving conflicts
   - staging resolution
   - completing merge
   - aborting merge
5. Rebase and history manipulation
   - `rebase`
   - interactive rebase
   - squash
   - reorder commits
   - amend
   - cherry-pick
6. Git recovery and debugging
   - `reflog`
   - reset vs revert
   - recovering lost commits
   - stash
   - `.gitignore`
   - inspecting history with `log` / `diff`

Skill 2 — Linux

Preparedness target: STRONG

Linux is extremely important for SDE/backend/system interviews. You don't need to become a Linux administrator, but you should be comfortable living in a Linux environment.

Subskills

1. Linux filesystem and navigation
    - `/`
    - `/home`
    - `/etc`
    - `/var`
    - `/tmp`
    - absolute vs relative paths
    - `pwd`
    - `cd`
    - `ls`
    - `find`
2. File and directory manipulation
    - `cp`
    - `mv`
    - `rm`
    - `mkdir`
    - `touch`
    - symbolic links
    - hard links
    - file metadata

3. Permissions and ownership
    - read/write/execute
    - user/group/other
    - `chmod`
    - `chown`
    - `umask`
    - permission notation
4. Processes and jobs
    - process IDs
    - parent/child processes
    - foreground/background processes
    - `ps`
    - `top`
    - `kill`
    - signals
    - `SIGTERM`
    - `SIGKILL`
    - process states
5. Pipes and shell composition
    - stdin
    - stdout
    - stderr
    - pipes
    - redirection
    - `|`
    - `>`

- `>>`
- `2>`
- command chaining

6. Text processing and searching
   - `grep`
   - `awk`
   - `sed`
   - `sort`
   - `uniq`
   - `cut`
   - `head`
   - `tail`
   - `wc`
7. Shell scripting
   - variables
   - arguments
   - conditionals
   - loops
   - functions
   - exit codes
   - command substitution
   - environment variables
8. Networking and socket inspection
   - `netstat`
   - `ss`
   - listening ports

- established connections
- `TIME_WAIT`
- connection inspection
- port/process mapping

9. Resource monitoring
    - CPU usage
    - memory usage
    - disk usage
    - `df`
    - `du`
    - load average
    - I/O basics
10. Linux engineering workflow
    - SSH
    - environment configuration
    - logs
    - package management
    - services
    - environment variables
    - troubleshooting applications on Linux

Skill 3 — Docker

Preparedness target: WORKING → STRONG

Docker appears repeatedly in your project evidence, so you need enough depth to explain not just commands but why containerization works.

Subskills

1. Container fundamentals
    - image vs container
    - container lifecycle
    - isolation
    - immutable images
    - container processes
2. Dockerfiles
    - base images
    - `FROM`
    - `RUN`
    - `COPY`
    - `CMD`
    - `ENTRYPOINT`
    - `ENV`
    - `EXPOSE`
    - build context
3. Image construction
    - image layers
    - layer caching
    - image tags
    - image size
    - reproducible builds
    - multi-stage builds
4. Container lifecycle and management
    - `run`

- `start`
- `stop`
- `restart`
- `exec`
- `logs`
- `inspect`
- container cleanup

5. Docker networking
    - bridge networks
    - container-to-container communication
    - port mapping
    - host networking
    - service discovery
6. Docker volumes and persistence
    - volumes
    - bind mounts
    - persistent data
    - database containers
    - ephemeral vs persistent storage
7. Docker Compose
    - multi-container applications
    - services
    - dependencies
    - networks
    - volumes
    - environment configuration

8. Docker production/debugging concepts
    - health checks
    - resource limits
    - container logs
    - crash recovery
    - graceful shutdown
    - containerized testing

Skill 4 — REST APIs

Preparedness target: STRONG

This should be second nature for you because backend development appears in several of your resume variants.

Subskills

1. HTTP fundamentals
    - request/response model
    - HTTP methods
    - URL
    - headers
    - body
    - status codes
2. HTTP methods and semantics
    - GET
    - POST
    - PUT
    - PATCH

- DELETE
- idempotency
- safe methods

3. HTTP status codes
    - 2xx
    - 3xx
    - 4xx
    - 5xx
    - choosing appropriate status codes
4. REST resource modelling
    - resources
    - collections
    - nested resources
    - resource identifiers
    - URI design
    - statelessness
5. Request validation and error handling
    - input validation
    - malformed requests
    - validation errors
    - consistent error responses
    - error schemas
    - exception handling
6. API pagination/filtering/sorting
    - offset pagination
    - cursor pagination

- filtering
- sorting
- search
- limits

7. REST API security fundamentals
    - authentication
    - authorization
    - JWT
    - API keys
    - HTTPS
    - CORS
    - rate limiting

Skill 5 — API Design

Preparedness target: STRONG

This overlaps with REST but goes one level above it: **how do you design an API that other engineers can actually depend upon?**

Subskills

1. Resource and domain modelling
    - identifying resources
    - relationships
    - ownership
    - lifecycle
    - resource boundaries
2. API contract design

- request schemas
- response schemas
- field naming
- optional vs required fields
- consistent contracts

3. API versioning
    - URL versioning
    - header versioning
    - backward compatibility
    - breaking changes
    - migration strategies
4. Error contract design
    - structured errors
    - error codes
    - human-readable messages
    - machine-readable errors
    - validation errors
5. Pagination and large-result APIs
    - offset pagination
    - cursor pagination
    - stable ordering
    - pagination consistency
    - performance implications
6. Idempotency and retries
    - idempotent operations
    - idempotency keys

- duplicate requests
- retry safety
- exactly-once illusions

7. API authentication and authorization
    - authentication
    - authorization
    - roles
    - permissions
    - JWT
    - OAuth/OIDC concepts
8. API reliability
    - timeouts
    - retries
    - rate limiting
    - circuit breakers
    - graceful degradation
    - dependency failures
9. API scalability and performance
    - caching
    - connection pooling
    - batching
    - asynchronous processing
    - payload size
    - compression
    - database interaction

Skill 6 — Flask

Preparedness target: WORKING

Flask is relatively straightforward. You don't need an enormous Flask-specific syllabus because the underlying concepts matter more than framework trivia.

Subskills

1. Flask application structure
    - application object
    - routes
    - request handling
    - configuration
    - blueprints
2. Flask request/response lifecycle
    - request object
    - response object
    - query parameters
    - path parameters
    - JSON bodies
    - headers
3. Flask middleware and hooks
    - middleware concepts
    - `before_request`
    - `after_request`
    - request context
    - application context
4. Flask database integration

- database connections
- ORM integration
- transactions
- connection lifecycle

5. Flask authentication and API security
    - JWT integration
    - authorization
    - validation
    - CORS
    - secure configuration
6. Flask testing and production basics
    - test client
    - API tests
    - error handlers
    - logging
    - production WSGI concepts

Skill 7 — FastAPI

Preparedness target: STRONG

FastAPI appears in your Search Engine and ML/NLP project evidence, so this deserves considerably more depth than Flask.

Subskills

1. FastAPI application architecture
    - application instance
    - routers

- route handlers
- modular architecture
- configuration

2. Pydantic models
    - request models
    - response models
    - validation
    - nested models
    - optional fields
    - serialization
3. Dependency injection
    - dependencies
    - reusable dependencies
    - dependency chains
    - authentication dependencies
    - database dependencies
4. Path/query/body/header parameters
    - path parameters
    - query parameters
    - request bodies
    - headers
    - cookies
5. Async FastAPI
    - `async`
    - `await`
    - asynchronous endpoints

- blocking vs non-blocking operations
- event-loop interaction

6. FastAPI authentication
    - OAuth2 concepts
    - JWT
    - bearer tokens
    - token validation
    - protected routes
7. FastAPI error handling
    - HTTP exceptions
    - custom exception handlers
    - validation errors
    - consistent API responses
8. FastAPI database integration
    - connection management
    - SQL queries
    - ORM integration
    - transactions
    - pooling
9. FastAPI testing
    - test client
    - endpoint testing
    - dependency overrides
    - integration tests
    - authentication tests
10. FastAPI production architecture

- ASGI
- Uvicorn
- workers
- reverse proxies
- deployment
- performance considerations

Skill 8 — Node.js

Preparedness target: STRONG

Node is important because it gives you another backend programming model and is highly relevant to full-stack/SDE roles.

Subskills

1. Node.js runtime fundamentals
    - V8
    - Node runtime
    - modules
    - CommonJS
    - ES modules
    - npm
2. Event loop
    - call stack
    - event loop
    - callback queue
    - microtasks
    - timers
    - I/O callbacks

3. Asynchronous programming
    - callbacks
    - Promises
    - async/await
    - Promise chaining
    - error propagation
    - parallel async operations
4. Node.js HTTP servers
    - HTTP module
    - request handling
    - response handling
    - headers
    - status codes
    - routing
5. Express-style backend architecture
    - routes
    - middleware
    - controllers
    - services
    - error middleware
    - request lifecycle
6. Node.js streams
    - readable streams
    - writable streams
    - piping
    - backpressure

- streaming large data

7. Node.js networking
    - TCP concepts
    - sockets
    - HTTP keep-alive
    - connection management
    - network errors
8. Node.js concurrency model
    - single-threaded JavaScript execution
    - asynchronous I/O
    - libuv
    - worker threads
    - CPU-bound workloads
9. npm/package management
    - package.json
    - dependencies
    - devDependencies
    - semantic versioning
    - lockfiles
    - scripts
10. Node.js security
    - input validation
    - authentication
    - authorization
    - secrets
    - dependency vulnerabilities

- CORS

11. Node.js database integration
    - database drivers
    - connection pooling
    - transactions
    - parameterized queries
    - ORM concepts
12. Node.js production engineering
    - environment configuration
    - logging
    - graceful shutdown
    - process management
    - health checks
    - error handling

Skill 9 — React

Preparedness target: WORKING → STRONG

React is useful for your full-stack profile, but I would deliberately avoid going absurdly deep here compared with DSA, DBMS, OS, networking, backend and systems.

Subskills

1. React component model
    - functional components
    - component composition
    - props
    - children

- reusable components

2. React state
   - local state
   - `useState`
   - state updates
   - derived state
   - lifting state
3. React effects
   - `useEffect`
   - dependency arrays
   - cleanup
   - side effects
   - common effect mistakes
4. React hooks
   - `useState`
   - `useEffect`
   - `useMemo`
   - `useCallback`
   - `useRef`
   - custom hooks
5. Component communication
   - parent → child
   - child → parent
   - callbacks
   - shared state
   - context

6. Forms and user input
    - controlled components
    - uncontrolled components
    - validation
    - form submission
    - error states
7. Conditional and list rendering
    - conditional UI
    - mapping collections
    - keys
    - empty/loading/error states
8. React API integration
    - HTTP requests
    - loading state
    - error state
    - response handling
    - authentication tokens
9. React routing
    - routes
    - nested routes
    - route parameters
    - protected routes
    - navigation
10. React Context
    - context creation
    - providers

- consumers
- avoiding unnecessary global state

11. Redux fundamentals
    - store
    - actions
    - reducers
    - selectors
    - dispatch
12. React performance
    - unnecessary renders
    - memoization
    - `React.memo`
    - `useMemo`
    - `useCallback`
    - list rendering performance
13. React architecture
    - component boundaries
    - container/presentational concepts
    - reusable components
    - feature-based organization
    - separation of concerns
14. React error/loading UX
    - loading states
    - error states
    - retry
    - optimistic UI

- fallback UI

15. Frontend security fundamentals
    - XSS
    - token storage
    - CSRF concepts
    - CORS
    - input sanitization
16. React testing fundamentals
    - component testing
    - user interaction testing
    - mocking API calls
    - integration testing
17. Frontend build/deployment basics
    - Vite
    - development vs production builds
    - environment variables
    - static assets
    - deployment

Skill 10 — TypeScript

Preparedness target: WORKING

TypeScript deserves enough depth to make your React/backend code genuinely typed, but again, this shouldn't consume the same time as DSA or CS fundamentals.

Subskills

1. TypeScript primitive and object types

    - string
    - number
    - boolean
    - arrays
    - tuples
    - objects
    - `null`
    - `undefined`
2. Interfaces and type aliases
    - interfaces
    - type aliases
    - optional properties
    - readonly properties
    - extending types
3. Union and intersection types
    - unions
    - intersections
    - discriminated unions
    - narrowing
4. Type narrowing
    - `typeof`
    - `instanceof`
    - `in`
    - custom type guards
    - control-flow analysis
5. Functions and typing

- parameter types
- return types
- optional parameters
- default parameters
- function types
- callbacks

6. Generics
   - generic functions
   - generic interfaces
   - generic classes
   - constraints
   - reusable type-safe code
7. Utility types
   - `Partial`
   - `Required`
   - `Pick`
   - `Omit`
   - `Record`
   - `Readonly`
   - `ReturnType`
8. Classes and OOP in TypeScript
   - classes
   - access modifiers
   - inheritance
   - abstract classes
   - interfaces

- implementations

9. TypeScript with async code
    - Promise types
    - async/await
    - typed API responses
    - error typing
    - generic API wrappers
10. TypeScript with React
    - typed props
    - typed state
    - event types
    - refs
    - context
    - component interfaces
11. TypeScript with APIs
    - request types
    - response types
    - API contracts
    - runtime validation vs compile-time typing
12. Advanced type concepts
    - `keyof`
    - indexed access types
    - conditional types
    - mapped types
    - `infer`
13. Modules and configuration

- imports/exports
- module resolution
- `tsconfig`
- compiler options
- strict mode

14. Type safety vs runtime safety
    - compile-time checking
    - runtime validation
    - unsafe casts
    - `any`
    - `unknown`
    - `never`
15. TypeScript engineering practices
    - strict typing
    - avoiding `any`
    - shared types
    - maintainable type definitions
    - API contract consistency

---

11. Java

Preparedness target: STRONG

Because Java appears in your backend and database projects, I would go fairly deep here. You don't need JVM-engineer depth, but you should be comfortable discussing Java in an SDE interview.

Subskills

1. Java syntax and fundamentals
    - variables

- primitive types
- operators
- control flow
- arrays
- methods

2. Object-oriented programming
    - classes
    - objects
    - encapsulation
    - inheritance
    - polymorphism
    - abstraction
3. Interfaces and abstract classes
    - interfaces
    - abstract classes
    - default methods
    - interface-based design
    - composition vs inheritance
4. Java memory model basics
    - stack
    - heap
    - references
    - object allocation
    - garbage collection concept
5. Strings and immutability
    - String

- StringBuilder
- StringBuffer
- immutability
- string pool

6. Collections framework
    - List
    - ArrayList
    - LinkedList
    - Set
    - HashSet
    - Map
    - HashMap
    - TreeMap
    - Queue
    - Deque
    - PriorityQueue
7. Hashing and HashMap internals
    - hash functions
    - buckets
    - collisions
    - equals/hashCode contract
    - resizing
    - load factor
8. Generics
    - generic classes
    - generic methods

- bounded types
- wildcards
- covariance/contravariance concepts

9. Exception handling
    - checked exceptions
    - unchecked exceptions
    - try/catch/finally
    - custom exceptions
    - exception propagation
10. Java functional programming
    - lambdas
    - functional interfaces
    - method references
    - streams
    - map/filter/reduce
11. Java I/O
    - byte streams
    - character streams
    - buffering
    - file operations
    - serialization concepts
12. Multithreading
    - Thread
    - Runnable
    - ExecutorService
    - thread pools

- synchronization
- locks

13. Concurrency utilities
    - synchronized
    - ReentrantLock
    - Atomic classes
    - ConcurrentHashMap
    - BlockingQueue
    - CountDownLatch
    - Semaphore
14. JVM basics
    - bytecode
    - JVM
    - JRE
    - JDK
    - class loading
    - JIT compilation
15. Garbage collection
    - garbage collection
    - reachability
    - generational collection
    - GC pauses
    - basic collector concepts
16. Java performance
    - object allocation
    - memory usage

- profiling basics
- thread contention
- performance bottlenecks

17. Java 21 awareness
    - modern Java features
    - records
    - pattern matching
    - virtual threads
    - modern concurrency concepts

---

12. C++

Preparedness target: STRONG / VERY STRONG

This is one of your most important skills because of your DSA/CP background and C++ TA experience.

Subskills

1. C++ fundamentals
    - syntax
    - primitive types
    - control flow
    - functions
    - arrays
    - structs
    - enums
2. Pointers and references
    - pointers

- references
- pointer arithmetic
- null pointers
- pointer/reference differences
- const correctness

3. Memory management
    - stack vs heap
    - new/delete
    - memory leaks
    - dangling pointers
    - double free
    - ownership
4. RAII
    - resource ownership
    - constructors/destructors
    - deterministic cleanup
    - exception safety
5. Smart pointers
    - unique_ptr
    - shared_ptr
    - weak_ptr
    - ownership semantics
    - reference counting
    - cyclic references
6. OOP in C++
    - classes

- encapsulation
- inheritance
- virtual functions
- abstract classes
- polymorphism

7. STL containers
    - vector
    - deque
    - list
    - set
    - unordered_set
    - map
    - unordered_map
    - priority_queue
    - stack
    - queue
8. STL algorithms
    - sort
    - lower_bound
    - upper_bound
    - binary_search
    - accumulate
    - min/max
    - heap operations
9. Iterators
    - iterator categories

- iterator invalidation
- begin/end
- reverse iterators
- iterator-based algorithms

10. Templates
    - function templates
    - class templates
    - template specialization
    - generic programming
11. Move semantics
    - lvalues
    - rvalues
    - move constructors
    - move assignment
    - std::move
    - copy vs move
12. Copy semantics
    - copy constructor
    - copy assignment
    - deep copy
    - shallow copy
    - Rule of Three
    - Rule of Five
    - Rule of Zero
13. Lambda functions
    - lambda syntax

- captures
- mutable lambdas
- generic lambdas
- lambdas with STL algorithms

14. Exception safety
    - exceptions
    - stack unwinding
    - RAII
    - exception guarantees
    - noexcept
15. C++ concurrency
    - std::thread
    - mutex
    - lock_guard
    - unique_lock
    - condition_variable
    - atomics
    - futures
16. C++ compilation model
    - preprocessing
    - compilation
    - assembly
    - linking
    - header files
    - translation units
    - static vs dynamic linking

17. C++ performance
    - cache locality
    - allocations
    - copies
    - move semantics
    - object lifetime
    - profiling basics
18. C++ debugging
    - compiler errors
    - runtime errors
    - segmentation faults
    - GDB basics
    - sanitizers
    - memory errors

---

13. Python

Preparedness target: STRONG

Python should be one of your easiest languages to become productive in, and it's central to your ML and search-engine work.

Subskills

1. Python syntax and fundamentals
    - variables
    - types
    - conditionals
    - loops

- functions

2. Python data structures
    - list
    - tuple
    - set
    - dictionary
    - deque
    - heapq
3. Functions
    - positional arguments
    - keyword arguments
    - default arguments
    - *args
    - **kwargs
    - closures
4. Python comprehensions
    - list comprehensions
    - dictionary comprehensions
    - set comprehensions
    - generator expressions
5. Iterators and generators
    - iterable
    - iterator
    - yield
    - generators
    - lazy evaluation

6. Object-oriented Python
    - classes
    - inheritance
    - polymorphism
    - properties
    - dunder methods
7. Exceptions
    - try/except
    - finally
    - custom exceptions
    - exception hierarchy
8. Modules and packages
    - imports
    - modules
    - packages
    - `__init__.py`
    - virtual environments
9. Python environments and dependency management
    - venv
    - pip
    - requirements.txt
    - dependency versions
    - reproducibility
10. Python performance
    - mutable vs immutable objects
    - copying

- generators
- profiling basics
- algorithmic complexity

11. Python concurrency awareness
    - threading
    - multiprocessing
    - asyncio
    - GIL
    - I/O-bound vs CPU-bound workloads

14. SQL

Preparedness target: VERY STRONG

This deserves significant depth. SQL is one of the highest-ROI interview skills in your entire list.

Subskills

1. Relational model
    - tables
    - rows
    - columns
    - primary keys
    - foreign keys
    - relationships
2. SELECT fundamentals
    - SELECT
    - WHERE
    - ORDER BY

- LIMIT
- DISTINCT

3. Filtering and predicates
    - AND/OR
    - IN
    - BETWEEN
    - LIKE
    - NULL
    - IS NULL
    - CASE
4. Aggregation
    - COUNT
    - SUM
    - AVG
    - MIN
    - MAX
    - GROUP BY
    - HAVING
5. Joins
    - INNER JOIN
    - LEFT JOIN
    - RIGHT JOIN
    - FULL JOIN
    - CROSS JOIN
    - self joins
6. Subqueries

- scalar subqueries
- correlated subqueries
- EXISTS
- NOT EXISTS
- IN subqueries

7. Common Table Expressions
    - CTEs
    - multiple CTEs
    - recursive CTE awareness
    - readability vs performance
8. Window functions
    - OVER
    - PARTITION BY
    - ORDER BY
    - ROW_NUMBER
    - RANK
    - DENSE_RANK
    - LAG
    - LEAD
9. SQL set operations
    - UNION
    - UNION ALL
    - INTERSECT
    - EXCEPT
10. Data modification
    - INSERT

- UPDATE
- DELETE
- UPSERT
- conflict handling

11. Constraints
    - PRIMARY KEY
    - FOREIGN KEY
    - UNIQUE
    - NOT NULL
    - CHECK
    - referential integrity
12. Transactions
    - BEGIN
    - COMMIT
    - ROLLBACK
    - atomicity
    - transaction boundaries
13. Isolation levels
    - read uncommitted
    - read committed
    - repeatable read
    - serializable
    - dirty reads
    - non-repeatable reads
    - phantom reads
14. SQL indexing concepts

- indexes
- B-trees
- composite indexes
- selectivity
- covering indexes
- index trade-offs

15. Query optimization
    - query plans
    - EXPLAIN
    - EXPLAIN ANALYZE
    - joins and indexes
    - avoiding unnecessary scans
16. Database normalization
    - 1NF
    - 2NF
    - 3NF
    - BCNF
    - denormalization
17. Advanced SQL interview patterns
    - top-K
    - ranking
    - running totals
    - gaps and islands
    - duplicate detection
    - hierarchical queries
    - time-series queries

15. PostgreSQL

Preparedness target: STRONG

This deserves more than MySQL because it appears repeatedly in your actual project work.

Subskills

1. PostgreSQL architecture
    - server
    - database
    - schema
    - tables
    - roles
    - connections
2. PostgreSQL data types
    - numeric
    - text
    - boolean
    - timestamps
    - arrays
    - JSON/JSONB
3. PostgreSQL schema design
    - schemas
    - constraints
    - relationships
    - foreign keys

- cascading behavior

4. PostgreSQL indexes
    - B-tree
    - Hash
    - GIN
    - GiST
    - composite indexes
    - partial indexes
    - expression indexes
5. PostgreSQL query planning
    - planner
    - cost estimates
    - sequential scans
    - index scans
    - bitmap scans
    - join strategies
6. EXPLAIN and EXPLAIN ANALYZE
    - execution plans
    - actual vs estimated rows
    - execution time
    - scan selection
    - identifying bottlenecks
7. PostgreSQL transactions
    - MVCC
    - snapshots
    - isolation

- locking
- concurrent transactions

8. PostgreSQL connection management
    - connections
    - connection pooling
    - pool sizing
    - idle connections
    - connection exhaustion
9. PostgreSQL performance
    - indexes
    - statistics
    - VACUUM
    - ANALYZE
    - query optimization
    - table bloat awareness
10. PostgreSQL JSON/JSONB
    - JSON storage
    - JSONB
    - querying nested fields
    - indexing JSONB
    - trade-offs
11. PostgreSQL extensions
    - extension architecture
    - pgvector
    - HypoPG
    - extension use cases

12. PostgreSQL production concerns
    - backups
    - replication awareness
    - migrations
    - connection limits
    - monitoring
    - failure recovery

---

16. MySQL

Preparedness target: WORKING

You don't need PostgreSQL-level depth here because your project evidence is much stronger around PostgreSQL.

Subskills

1. MySQL architecture
    - databases
    - tables
    - users
    - connections
2. MySQL SQL fundamentals
    - SELECT
    - JOIN
    - GROUP BY
    - subqueries
    - window functions
3. MySQL storage engines

- InnoDB
- transactional storage
- basic engine differences

4. MySQL indexes
   - B-tree
   - composite indexes
   - selectivity
   - index usage
5. MySQL transactions
   - ACID
   - isolation
   - locks
   - commit/rollback
6. MySQL query optimization
   - EXPLAIN
   - query plans
   - indexes
   - joins
   - common performance problems

---

17. Redis

Preparedness target: WORKING

Redis is particularly useful for backend/system-design interviews.

Subskills

1. Redis fundamentals

- in-memory datastore
- key-value model
- commands
- data persistence awareness

2. Redis data structures
    - strings
    - hashes
    - lists
    - sets
    - sorted sets
3. TTL and expiration
    - EXPIRE
    - TTL
    - lazy expiration
    - active expiration
    - stale data
4. Redis caching patterns
    - cache-aside
    - read-through
    - write-through
    - cache invalidation
5. Redis eviction
    - memory limits
    - LRU
    - LFU
    - eviction policies

6. Redis persistence
    - RDB
    - AOF
    - durability trade-offs
7. Redis atomicity
    - atomic commands
    - transactions
    - MULTI/EXEC
    - Lua scripting awareness
8. Redis concurrency/distributed use
    - race conditions
    - distributed locks
    - counters
    - rate limiting
    - pub/sub awareness
9. Redis system-design trade-offs
    - cache vs database
    - consistency
    - durability
    - memory cost
    - cache stampede

18. Spring Boot

Preparedness target: STRONG

This is important because your PostgreSQL Index Advisor uses Java 21 + Spring Boot.

Subskills

1. Spring Boot fundamentals
    - application structure
    - auto-configuration
    - starters
    - application lifecycle
2. Dependency Injection
    - IoC
    - dependency injection
    - constructor injection
    - bean lifecycle
3. Controllers
    - REST controllers
    - routes
    - request mapping
    - request parameters
    - request bodies
4. Service architecture
    - controllers
    - services
    - repositories
    - separation of concerns
5. Spring configuration
    - application properties
    - environment variables
    - profiles

- configuration classes

6. Database integration
    - JDBC
    - DataSource
    - connection pooling
    - transactions
7. JPA/Hibernate fundamentals
    - entities
    - repositories
    - relationships
    - lazy/eager loading
    - N+1 problem
8. Spring transactions
    - @Transactional
    - transaction boundaries
    - rollback
    - isolation concepts
9. Validation and error handling
    - validation annotations
    - DTO validation
    - exception handlers
    - consistent error responses
10. Spring Boot testing
    - unit tests
    - integration tests
    - controller tests

- repository tests
- mocking

11. Spring Boot security awareness
    - authentication
    - authorization
    - filters
    - JWT
    - Spring Security concepts
12. Spring Boot production engineering
    - logging
    - Actuator
    - health endpoints
    - configuration
    - deployment
    - performance

---

19. Redux

Preparedness target: WORKING

Redux is less important than React itself, so we keep this compact.

Subskills

1. Redux architecture
    - store
    - state
    - actions
    - reducers
    - dispatch
2. Reducers
    - pure functions
    - immutable updates
    - state transitions

3. Actions
   - action objects
   - action creators
   - payloads
4. Selectors
   - reading state
   - derived state
   - memoized selectors
5. Redux Toolkit
   - configureStore
   - createSlice
   - createAsyncThunk
   - simplified immutable updates
6. Async state management
   - loading
   - success
   - failure
   - API state
7. Redux architecture decisions
   - local vs global state
   - normalized state
   - avoiding unnecessary Redux
   - state ownership

---

20. Vite

Preparedness target: BASIC / WORKING

Vite is tooling rather than a major interview domain.

Subskills

1. Vite fundamentals
   - development server

- build system
- project structure

2. Vite configuration
   - `vite.config`
   - plugins
   - aliases
   - environment configuration
3. Development vs production
   - hot module replacement
   - production builds
   - static assets
   - build output
4. Environment variables
   - `.env`
   - development variables
   - production variables
   - client-side exposure considerations
5. Vite with React/TypeScript
   - React integration
   - TypeScript integration
   - routing
   - API configuration
6. Vite deployment basics
   - build command
   - deployment artifacts
   - static hosting
   - debugging production builds

21. Multithreading

Preparedness target: STRONG

This is a major SDE/system interview topic and should be much deeper than framework-specific skills.

Subskills

1. Process vs thread
    - process
    - thread
    - address-space differences
    - resource sharing
    - creation overhead
2. Thread lifecycle
    - creation
    - running
    - blocking
    - waiting
    - termination
    - joining
3. Thread creation
    - thread APIs
    - Runnable/callable concepts
    - detached vs joined threads
4. Thread pools
    - worker threads
    - task queues
    - fixed thread pools
    - dynamic pools
    - pool sizing
5. Shared state
    - shared memory

- mutable state
- race conditions
- thread safety

6. Race conditions
    - data races
    - read-modify-write
    - lost updates
    - timing-dependent bugs
7. Mutual exclusion
    - mutex
    - locks
    - critical sections
    - lock ownership
8. Lock types
    - mutex
    - reentrant locks
    - read/write locks
    - spinlocks awareness
9. Atomic operations
    - atomicity
    - atomic variables
    - compare-and-swap
    - lock-free concepts
10. Condition variables
    - waiting
    - notification

- producer-consumer
- spurious wakeups

11. Deadlocks
    - circular wait
    - hold and wait
    - mutual exclusion
    - no preemption
    - deadlock prevention
    - deadlock detection
12. Starvation and fairness
    - starvation
    - unfair scheduling
    - priority inversion
    - fairness
13. Thread synchronization patterns
    - producer-consumer
    - readers-writers
    - barriers
    - semaphores
    - latches
14. Thread safety
    - immutable objects
    - synchronized access
    - confinement
    - safe publication
15. Thread communication

- shared memory
- queues
- channels
- condition variables

16. CPU-bound vs I/O-bound concurrency
    - parallelism
    - concurrency
    - blocking I/O
    - asynchronous I/O
17. Thread scheduling
    - scheduler
    - context switching
    - priorities
    - preemption
    - scheduling overhead
18. Multithreading performance
    - lock contention
    - false sharing awareness
    - context-switch overhead
    - scalability
    - Amdahl's Law
19. Debugging concurrent programs
    - race reproduction
    - thread dumps
    - deadlock detection
    - logging

- concurrent stress testing

20. Language-specific concurrency
    - Java concurrency
    - C++ concurrency
    - Python threading/GIL awareness
    - async programming distinction

---

22. Concurrency

Preparedness target: STRONG

Multithreading is about threads; concurrency is the broader reasoning framework. This distinction matters a lot in interviews.

Subskills

1. Concurrency vs parallelism
- concurrency
- parallelism
- multitasking
- multicore execution

1. Shared-memory concurrency
- shared state
- synchronization
- atomicity
- visibility

1. Message-passing concurrency
- queues
- channels

- actor-style communication
- producer-consumer

1. Synchronization strategies

- locks
- atomics
- semaphores
- condition variables
- barriers

1. Memory visibility

- stale reads
- happens-before
- memory barriers
- visibility guarantees

1. Atomicity

- atomic operations
- compound operations
- compare-and-swap

1. Lock contention

- contention
- critical-section size
- lock granularity
- throughput impact

1. Deadlock analysis

- wait-for graphs
- lock ordering
- timeout strategies

- prevention

1. Lock-free/wait-free awareness

- CAS
- lock-free algorithms
- progress guarantees

1. Async concurrency

- event loops
- futures
- promises
- async/await
- callbacks

1. Concurrent data structures

- concurrent maps
- concurrent queues
- blocking queues
- thread-safe collections

1. Concurrency testing

- stress testing
- randomized scheduling
- race detection
- deterministic reproduction

1. Concurrency design trade-offs

- correctness vs performance
- lock complexity
- throughput vs latency
- scalability

23. TCP Sockets

Preparedness target: STRONG

This is particularly important because you've actually implemented a distributed key-value cache using TCP.

Subskills

1. Network layering basics

- application layer
- transport layer
- IP vs TCP
- sockets as programming abstraction

1. Socket lifecycle

- socket creation
- bind
- listen
- accept
- connect
- send
- receive
- close

1. TCP connection establishment

- three-way handshake
- SYN
- SYN-ACK
- ACK

1. TCP connection termination

- FIN
- ACK
- connection teardown
- TIME_WAIT

1. TCP reliability

- sequence numbers
- acknowledgements
- retransmission
- duplicate packets
- ordered delivery

1. TCP flow control

- receive window
- sender rate
- receiver capacity

1. TCP congestion control

- congestion window
- slow start
- congestion avoidance
- retransmission awareness

1. TCP byte-stream semantics

- no message boundaries
- partial reads
- partial writes
- application-level framing

1. Socket blocking modes

- blocking sockets

- non-blocking sockets
- timeouts
- I/O multiplexing awareness

1. Connection management

- persistent connections
- keep-alive
- connection pooling
- connection limits

1. TCP failures

- connection reset
- timeout
- broken pipe
- half-open connections
- network failure

1. Ports and sockets

- IP address
- port
- socket tuple
- ephemeral ports
- port exhaustion

1. TIME_WAIT

- why it exists
- socket reuse
- ephemeral-port exhaustion
- implications for high-throughput clients

1. TCP server architecture

- single-threaded server
- thread-per-connection
- thread pool
- event-driven server

1. TCP protocol design

- message framing
- serialization
- request/response protocols
- versioning
- error handling

---

24. Client-Server Architecture

Preparedness target: WORKING → STRONG

Subskills

1. Client-server model

- client
- server
- request
- response
- responsibilities

1. Stateless vs stateful servers

- session state
- server-side state
- client-side state
- scalability implications

1. Request-response architecture
- synchronous requests
- asynchronous requests
- timeouts
- retries
1. Persistent connections
- keep-alive
- connection reuse
- pooling
1. Serialization
- JSON
- binary serialization
- Protocol Buffers
- compatibility
1. Service boundaries
- responsibility separation
- APIs
- service contracts
- dependency boundaries
1. Failure handling
- timeouts
- retries
- partial failures
- graceful degradation
1. Scaling client-server systems
- horizontal scaling

- load balancing
- stateless servers
- caching

1. Client-server security

- authentication
- authorization
- TLS
- secure communication

1. Architectural trade-offs

- latency
- throughput
- reliability
- complexity
- scalability

---

25. Caching

Preparedness target: STRONG

Caching is one of the highest-ROI system-design concepts for you.

Subskills

1. Why caching works

- locality
- latency reduction
- database load reduction
- throughput improvement

1. Cache hierarchy

- CPU cache awareness
- application cache
- distributed cache
- CDN awareness

1. Cache-aside

- cache lookup
- cache miss
- database lookup
- cache population

1. Read-through caching

- cache abstraction
- automatic loading
- cache misses

1. Write-through caching

- synchronous cache/database writes
- consistency
- latency trade-offs

1. Write-back caching

- deferred persistence
- dirty data
- durability risks

1. Cache invalidation

- explicit invalidation
- TTL
- stale data
- invalidation strategies

1. Eviction policies

- LRU
- LFU
- FIFO
- random eviction

1. TTL

- expiration
- lazy expiration
- active expiration
- refresh

1. Cache consistency

- stale reads
- write ordering
- consistency models
- invalidation races

1. Cache stampede

- thundering herd
- request coalescing
- jittered expiration
- locking

1. Cache penetration

- nonexistent-key attacks
- negative caching
- Bloom filters

1. Cache warming

- cold cache

- preloading
- warm-up strategies

1. Distributed caching

- Redis
- partitioning
- replication
- distributed cache failures

1. Cache sizing

- working set
- memory limits
- hit rate
- eviction pressure

1. Cache metrics

- hit rate
- miss rate
- latency
- memory utilization

1. Caching system-design decisions

- what to cache
- TTL selection
- consistency vs performance
- cache failure behavior

26. LRU Cache

Preparedness target: STRONG

You have implemented this, so you should be able to explain both the textbook implementation and your own implementation.

Subskills

1. LRU concept

- least recently used
- recency ordering
- eviction

1. Hash map + linked list design

- O(1) lookup
- O(1) insertion
- O(1) eviction

1. Cache operations

- GET
- PUT
- update existing key
- eviction

1. Doubly linked list implementation

- head
- tail
- node removal
- node insertion

1. Hash map internals

- key → node mapping
- synchronization with list

1. LRU correctness

- maintaining order
- updating access order
- capacity handling

1. LRU concurrency

- concurrent reads
- concurrent writes
- locks
- race conditions

1. LRU performance

- O(1) operations
- memory overhead
- cache locality

---

27. TTL Caching

Preparedness target: WORKING

Subskills

1. TTL fundamentals

- expiration timestamp
- TTL duration
- expired entries

1. Lazy expiration

- expiration on access
- stale memory

1. Active expiration

- background cleanup

- expiration scans

1. TTL refresh

- sliding expiration
- fixed expiration
- refresh-on-access

1. TTL design trade-offs

- freshness
- hit rate
- memory usage
- database load

1. TTL failure modes

- synchronized expiration
- stale data
- cache stampede

---

28. Performance Engineering

Preparedness target: STRONG

This is a particularly valuable differentiator because your projects contain concrete performance claims.

Subskills

1. Latency vs throughput

- latency
- throughput
- response time
- service rate

1. Percentile latency

- p50
- p90
- p95
- p99
- tail latency

1. Benchmark design

- workload definition
- warm-up
- steady state
- repetitions
- variance

1. Microbenchmarking

- isolated operations
- benchmark overhead
- compiler optimizations
- realistic workloads

1. Load testing

- concurrent clients
- request rates
- saturation
- throughput curves

1. Profiling

- CPU profiling
- memory profiling
- allocation profiling

- hot paths

1. Bottleneck identification

- CPU-bound
- memory-bound
- I/O-bound
- network-bound
- lock-bound

1. Memory performance

- allocations
- fragmentation
- cache locality
- memory bandwidth

1. CPU performance

- instruction cost
- branch prediction awareness
- cache behavior
- vectorization awareness

1. Concurrency performance

- contention
- scalability
- thread overhead
- synchronization cost

1. Database performance
   - query latency
   - indexes
   - connection pools
   - query plans

2. Network performance
   - bandwidth
   - latency
   - packet overhead
   - connection reuse
3. Performance regression testing
   - baseline
   - benchmark comparison
   - regression detection
   - reproducibility
4. Capacity planning
   - workload estimation
   - saturation point
   - headroom
   - scaling requirements
5. Performance trade-offs
   - latency vs throughput
   - memory vs CPU
   - consistency vs performance
   - complexity vs optimization
6. Interpreting benchmark results
   - statistical noise
   - outliers
   - confidence
   - median vs mean
   - reproducibility

29. Debugging

Preparedness target: STRONG

This is often underestimated. Strong debugging ability is extremely valuable in SDE interviews.

Subskills

1. Debugging methodology
   - reproduce
   - isolate
   - hypothesize
   - test
   - fix
   - verify
2. Reading errors
   - compiler errors
   - exceptions
   - stack traces
   - HTTP errors
   - database errors
3. Logging
   - log levels
   - structured logging
   - useful context
   - correlation IDs
   - avoiding sensitive data
4. Breakpoint debugging
   - breakpoints
   - stepping
   - watch expressions
   - call stacks
5. Memory debugging
   - leaks
   - dangling pointers
   - invalid memory access
   - use-after-free
6. Concurrency debugging
   - race conditions
   - deadlocks
   - timing-dependent bugs
   - thread dumps

7. Network debugging
   - connection failures
   - DNS awareness
   - port conflicts
   - packet/request inspection
8. Database debugging
   - slow queries
   - deadlocks
   - connection exhaustion
   - incorrect transactions
9. Production debugging
   - logs
   - metrics
   - traces
   - reproduction
   - safe mitigation
10. Performance debugging
    - profiling
    - bottleneck identification
    - benchmark comparison
    - regression analysis
11. Root-cause analysis
    - symptom vs cause
    - dependency analysis
    - five-whys
    - minimizing assumptions
12. Debugging distributed systems
    - partial failures
    - correlation IDs
    - distributed tracing
    - retry amplification
    - inconsistent state

13. Regression prevention
    - tests
    - regression tests
    - monitoring
    - assertions
    - postmortems

---

30. Database Indexing

Preparedness target: VERY STRONG

This is one of the strongest specialized areas in your resume. You should be able to go significantly deeper than the average SDE candidate.

Subskills

1. Why indexes exist
   - avoiding full scans
   - lookup acceleration
   - read/write trade-offs
2. B-tree fundamentals
   - balanced tree
   - ordered keys
   - logarithmic lookup
   - node structure
3. B-tree operations
   - search
   - insertion
   - deletion
   - splitting
   - rebalancing
4. B-tree index usage
   - equality
   - range queries

- ordering
- prefix matching

5. Composite indexes
   - multiple columns
   - column order
   - leftmost-prefix principle
   - equality + range
6. Index selectivity
   - cardinality
   - selectivity
   - high vs low selectivity
   - planner decisions
7. Covering indexes
   - index-only scans
   - included columns
   - avoiding table access
8. Partial indexes
   - conditional indexes
   - filtered datasets
   - use cases
9. Expression indexes
   - computed expressions
   - function-based indexing
   - query matching
10. Index scan types
    - index scan
    - index-only scan
    - bitmap index scan
    - sequential scan
11. Index maintenance
    - insert cost
    - update cost

    - delete cost
    - storage overhead
12. Redundant indexes
    - duplicate indexes
    - overlapping indexes
    - unnecessary indexes
13. Index prefix domination
    - prefix relationships
    - redundant composite indexes
    - domination pruning
14. Query planner interaction
    - cost estimates
    - statistics
    - selectivity estimates
    - planner choices
15. EXPLAIN-based index analysis
    - execution plans
    - scan selection
    - estimated vs actual cost
    - measuring improvements
16. Indexing trade-offs
    - read performance
    - write performance
    - memory/storage
    - maintenance
17. Index design methodology
    - workload analysis
    - query frequency
    - candidate generation
    - benchmarking
18. Hypothetical index evaluation
    - hypothetical indexes
    - planner simulation

- candidate screening
- validation

19. Index-set optimization
    - marginal utility
    - candidate combinations
    - greedy selection
    - redundant-index elimination
20. Production index engineering
    - migration impact
    - index creation cost
    - concurrent index creation awareness
    - operational safety
21. Index interview scenarios
    - why query ignores index
    - wrong composite-index order
    - too many indexes
    - low-selectivity index
    - index vs sequential scan

---

31. Query Optimization

Preparedness target: VERY STRONG

This pairs directly with your PostgreSQL Index Advisor project.

Subskills

1. Query execution lifecycle
    - parsing
    - planning
    - optimization
    - execution
2. Query plans
    - operators
    - scans

- joins
- aggregations
- sorts

3. EXPLAIN
   - estimated cost
   - estimated rows
   - plan structure
4. EXPLAIN ANALYZE
   - actual execution
   - actual rows
   - timing
   - loops
   - estimation errors
5. Sequential scans
   - when they are appropriate
   - full table scans
   - cost trade-offs
6. Index scans
   - index lookup
   - heap access
   - index-only scans
7. Bitmap scans
   - bitmap creation
   - bitmap heap scan
   - combining indexes
8. Join algorithms
   - nested-loop join
   - hash join
   - merge join
   - choosing join strategies
9. Join ordering
   - join cardinality

- intermediate results
- optimizer decisions

10. Sorting and aggregation
    - sort operations
    - hash aggregation
    - memory usage
    - disk spill
11. Query statistics
    - table statistics
    - column statistics
    - histograms
    - cardinality estimation
12. Query optimization techniques
    - indexing
    - predicate pushdown
    - avoiding unnecessary columns
    - query rewriting
    - reducing intermediate results
13. N+1 query problem
    - identifying N+1
    - joins
    - batching
    - eager loading
14. Query performance diagnosis
    - slow-query identification
    - bottleneck analysis
    - execution plans
    - benchmark methodology
15. Database connection effects
    - connection pooling
    - concurrency
    - pool saturation
    - query queueing

16. Query optimization trade-offs
    - index vs write cost
    - memory vs speed
    - query complexity
    - maintainability
17. Benchmark methodology
    - warm vs cold cache
    - repeated measurements
    - median
    - outliers
    - controlled workloads
18. Production query optimization
    - safe changes
    - regression testing
    - monitoring
    - rollback strategies

32. pgvector

Preparedness target: WORKING

This is specialized but directly relevant to your resume.

Subskills

1. Vector embeddings
    - embedding representation
    - dimensionality
    - semantic meaning
2. Vector storage
    - vector columns
    - dimensional constraints
    - PostgreSQL integration
3. Similarity metrics
    - cosine similarity

   - Euclidean distance
   - inner product
4. Vector similarity queries
   - nearest neighbours
   - ordering by distance
   - top-K retrieval
5. Vector indexes
   - approximate nearest-neighbour concepts
   - index trade-offs
   - recall vs speed
6. Semantic retrieval architecture
   - embedding generation
   - storage
   - query embedding
   - similarity search
   - ranking
7. Hybrid retrieval
   - keyword search
   - vector search
   - BM25
   - score fusion

---

33. HypoPG

Preparedness target: INTERVIEW-READY / RESUME-DEFENSIBLE

Subskills

1. Hypothetical indexes
   - what they are
   - why they are useful
   - planner simulation
2. Candidate index screening
   - candidate generation

   - hypothetical evaluation
   - cost comparison
3. EXPLAIN with hypothetical indexes
   - plan comparison
   - estimated improvement
   - candidate ranking
4. Screen-then-validate methodology
   - hypothetical screening
   - real index creation
   - EXPLAIN ANALYZE
   - validation
5. HypoPG limitations
   - estimates vs real performance
   - workload effects
   - storage considerations
   - concurrency considerations

---

34. HikariCP

Preparedness target: INTERVIEW-READY

Subskills

1. Connection pooling
   - why pooling exists
   - connection creation cost
   - pool reuse
2. Pool lifecycle
   - acquire
   - use
   - release
   - idle connections
3. Pool sizing
   - minimum pool

   - maximum pool
   - workload
   - database capacity
4. Pool exhaustion
   - waiting threads
   - timeouts
   - leaked connections
   - overload
5. HikariCP configuration
   - connection timeout
   - idle timeout
   - max lifetime
   - pool size
6. Concurrency issues
   - thread safety
   - races
   - connection lifecycle
   - concurrent load
7. Database pool/system interaction
   - application threads
   - pool
   - database connections
   - query latency
   - backpressure

---

35. Apache Kafka

Preparedness target: STRONG / INTERVIEW-READY

Kafka is highly valuable for backend/distributed-system roles and appears directly in your resume.

Subskills

1. Kafka architecture
   - brokers
   - topics
   - partitions
   - producers
   - consumers
2. Topics
   - topic creation
   - partitions
   - retention
   - message ordering
3. Partitions
   - partitioning
   - keys
   - ordering
   - parallelism
   - partition assignment
4. Producers
   - producing messages
   - keys
   - batching
   - acknowledgements
   - retries
5. Consumers
   - polling
   - processing
   - committing offsets
   - consumer lifecycle
6. Consumer groups
   - group membership
   - partition assignment
   - load balancing
   - consumer scaling

7. Offset management
    - offsets
    - commits
    - committed offsets
    - replay
    - offset reset
8. Delivery semantics
    - at-most-once
    - at-least-once
    - exactly-once concepts
    - duplicate processing
9. Ordering guarantees
    - partition-level ordering
    - global ordering limitations
    - keys and ordering
10. Kafka retention
    - time-based retention
    - size-based retention
    - log segments
    - replay
11. Fault tolerance
    - replication
    - leaders
    - followers
    - broker failures
    - recovery
12. Consumer rebalancing
    - joining/leaving consumers
    - partition reassignment
    - rebalance impact
13. Backpressure
    - producer pressure
    - consumer lag

- processing capacity
- buffering

14. Idempotent consumers
    - duplicate messages
    - deduplication
    - idempotent processing
15. Kafka performance
    - batching
    - compression
    - partition count
    - throughput
    - latency
16. Kafka failure scenarios
    - consumer crash
    - broker failure
    - duplicate processing
    - lost processing
    - lag
17. Kafka system-design patterns
    - event-driven architecture
    - streaming pipelines
    - decoupling services
    - asynchronous processing

36. Distributed Systems

Preparedness target: INTERVIEW-READY

This is one of the hardest domains in the entire placement syllabus. Don't try to become a distributed-systems researcher in 100 days. Become excellent at the **interview fundamentals**.

Subskills

1. Distributed-system fundamentals
   - multiple machines
   - network communication
   - independent failures
   - partial failure
2. Scalability
   - vertical scaling
   - horizontal scaling
   - load distribution
   - bottlenecks
3. Availability
   - uptime
   - redundancy
   - failover
   - graceful degradation
4. Reliability
   - failure detection
   - retries
   - recovery
   - redundancy
5. Consistency
   - strong consistency
   - eventual consistency
   - read-after-write
   - stale reads
6. CAP theorem
   - consistency
   - availability
   - partition tolerance
   - network partitions
7. Replication
   - primary-replica
   - synchronous replication

- asynchronous replication
- replication lag

8. Partitioning/sharding
   - horizontal partitioning
   - shard keys
   - range partitioning
   - hash partitioning
   - hotspots
9. Consistent hashing
   - hash ring
   - virtual nodes
   - node addition/removal
   - key redistribution
10. Load balancing
    - round robin
    - weighted routing
    - least connections
    - health checks
11. Distributed caching
    - cache nodes
    - partitioning
    - replication
    - consistency
12. Distributed transactions awareness
    - transaction boundaries
    - two-phase commit
    - distributed transaction costs
13. Idempotency
    - retries
    - duplicate requests
    - idempotency keys
    - exactly-once illusion

14. Message queues
    - asynchronous communication
    - producers
    - consumers
    - acknowledgements
    - retries
15. Backpressure
    - producer-consumer imbalance
    - queues
    - rate limiting
    - overload protection
16. Timeouts and retries
    - request timeout
    - retry policies
    - exponential backoff
    - jitter
    - retry storms
17. Failure modes
    - machine failure
    - network failure
    - service failure
    - partial failure
    - cascading failure
18. Circuit breakers
    - closed
    - open
    - half-open
    - failure thresholds
19. Service discovery
    - service registry
    - dynamic addresses
    - health checking

20. Distributed observability
    - logs
    - metrics
    - tracing
    - correlation IDs
21. Ordering
    - message ordering
    - timestamps
    - logical clocks awareness
    - partition-level ordering
22. Distributed-system design trade-offs
    - consistency vs availability
    - latency vs durability
    - simplicity vs scalability
    - synchronous vs asynchronous
23. Distributed-system interview patterns
    - URL shortener
    - rate limiter
    - notification system
    - distributed cache
    - message queue
    - chat system
    - feed system

---

37. Rust

Preparedness target: WORKING / RESUME-DEFENSIBLE

Rust is specialized. Because you actually have Rust/Tokio in your trading-engine project, however, you must be able to explain why you used it.

Subskills

1. Rust syntax
    - variables

   - functions
   - structs
   - enums
   - pattern matching
2. Ownership
   - ownership rules
   - moves
   - ownership transfer
   - scope
3. Borrowing
   - references
   - mutable references
   - borrowing rules
   - aliasing
4. Lifetimes
   - lifetime concept
   - lifetime annotations
   - reference validity
5. Traits
   - trait definitions
   - implementations
   - trait bounds
   - generic programming
6. Error handling
   - Result
   - Option
   - `?`
   - recoverable vs unrecoverable errors
7. Collections
   - Vec
   - HashMap
   - String
   - slices

8. Rust memory safety
   - no use-after-free
   - no data races
   - ownership-based safety
9. Rust concurrency
   - threads
   - Send
   - Sync
   - channels
   - shared state
10. Async Rust
    - futures
    - async/await
    - runtime concepts
    - Tokio awareness
11. Rust engineering
    - Cargo
    - crates
    - modules
    - testing
    - dependency management

38. Tokio

Preparedness target: RESUME-DEFENSIBLE

Subskills

1. Async runtime
   - runtime
   - task scheduling
   - asynchronous execution
2. Futures
   - future concept

   - polling
   - async execution
3. Async/await
   - async functions
   - awaiting futures
   - cooperative execution
4. Tokio tasks
   - spawning
   - task lifecycle
   - concurrency
5. Tokio channels
   - message passing
   - producer-consumer
   - bounded/unbounded channels
6. Async I/O
   - TCP
   - sockets
   - non-blocking I/O
   - asynchronous networking
7. Tokio concurrency architecture
   - tasks vs threads
   - blocking work
   - runtime workers

---

39. gRPC

Preparedness target: WORKING

Subskills

1. RPC fundamentals
   - remote procedure call
   - client/server model
   - service contracts

2. Protocol Buffers
   - messages
   - fields
   - serialization
   - schema
3. Service definitions
   - RPC methods
   - request/response types
   - generated code
4. gRPC communication
   - unary RPC
   - streaming RPC
   - client streaming
   - server streaming
   - bidirectional streaming
5. gRPC error handling
   - status codes
   - deadlines
   - cancellation
   - error propagation
6. gRPC performance
   - binary serialization
   - HTTP/2 awareness
   - connection reuse
   - latency
7. gRPC vs REST
   - contract-first APIs
   - serialization
   - browser compatibility
   - performance
   - use cases

40. Protocol Buffers

Preparedness target: WORKING

Subskills

1. Protobuf fundamentals
   - messages
   - fields
   - field numbers
   - scalar types
2. Serialization
   - binary encoding
   - serialization/deserialization
   - compact representation
3. Schema evolution
   - adding fields
   - removing fields
   - field numbering
   - backward compatibility
4. Compatibility
   - old clients/new servers
   - new clients/old servers
   - safe schema changes
5. Protobuf with gRPC
   - service definitions
   - generated clients
   - generated servers
   - shared contracts
6. Protobuf design practices
   - field naming
   - reserved fields
   - versioning
   - avoiding breaking changes

41. Keycloak

Preparedness target: WORKING / RESUME-DEFENSIBLE

You don't need to become a Keycloak administrator. Your goal is to understand what Keycloak did in your trading platform and be able to defend the authentication architecture.

Subskills

1. Identity and access management
    - identity
    - authentication
    - authorization
    - identity provider
    - access control
2. Keycloak architecture
    - Keycloak server
    - realms
    - clients
    - users
    - roles
3. Realms
    - realm concept
    - realm isolation
    - realm configuration
    - users within realms
4. Clients
    - client applications
    - client IDs

- client secrets
- public vs confidential clients

5. Users and roles
    - users
    - groups
    - roles
    - role assignment
    - role-based access
6. OIDC integration
    - OpenID Connect
    - identity tokens
    - access tokens
    - claims
    - discovery endpoints
7. OAuth2 integration
    - authorization server
    - resource server
    - client
    - access token
    - scopes
8. Authentication flows
    - authorization code flow
    - login
    - authorization
    - token exchange
    - refresh tokens

9. Token validation
    - JWT structure
    - signature
    - claims
    - issuer
    - audience
    - expiration
10. Application integration
    - frontend authentication
    - backend authentication
    - protected endpoints
    - token forwarding
11. Role-based authorization
    - realm roles
    - client roles
    - permissions
    - authorization decisions
12. Keycloak security considerations
    - token expiration
    - refresh tokens
    - secret management
    - HTTPS
    - least privilege
13. Authentication failure handling
    - invalid tokens
    - expired tokens

- missing credentials
- insufficient permissions

14. Keycloak architecture interview questions
    - why use Keycloak?
    - Keycloak vs implementing auth yourself
    - OAuth vs OIDC
    - access token vs ID token
    - authentication vs authorization

---

42. OpenID Connect (OIDC)

Preparedness target: STRONG

OIDC is more important conceptually than Keycloak itself. You should be able to explain exactly what happens when a user logs into your application.

Subskills

1. Authentication vs authorization
    - authentication
    - authorization
    - identity
    - permissions
2. OAuth 2.0 vs OIDC
    - OAuth as authorization
    - OIDC as authentication
    - identity layer
    - access tokens vs ID tokens
3. OIDC actors

- end user
- relying party/client
- authorization server
- resource server
- identity provider

4. Authorization Code Flow
    - authorization request
    - authorization code
    - token exchange
    - redirect URI
5. PKCE
    - code verifier
    - code challenge
    - authorization code interception
    - public clients
6. ID tokens
    - JWT
    - identity claims
    - subject
    - issuer
    - audience
    - expiration
7. Access tokens
    - bearer tokens
    - scopes
    - resource access

- expiration

8. Refresh tokens
    - token renewal
    - expiration
    - rotation awareness
    - revocation
9. OIDC discovery
    - discovery endpoint
    - provider metadata
    - authorization endpoint
    - token endpoint
    - JWKS
10. JWT fundamentals
    - header
    - payload
    - signature
    - claims
    - signing vs encryption
11. Token validation
    - signature verification
    - issuer validation
    - audience validation
    - expiration
    - nonce
12. Redirect URI security
    - exact redirect matching

- open redirect risks
- authorization-code leakage

13. OIDC security
    - CSRF/state
    - nonce
    - PKCE
    - token theft
    - secure storage
14. OIDC architecture questions
    - login flow
    - why ID token exists
    - why access token exists
    - OAuth vs OIDC
    - JWT vs opaque tokens

---

43. ClickHouse

Preparedness target: INTERVIEW-READY / RESUME-DEFENSIBLE

ClickHouse is specialized. You should understand **why it was used**, rather than trying to master every ClickHouse feature.

Subskills

1. OLTP vs OLAP
    - transactional workloads
    - analytical workloads
    - query patterns
    - latency vs throughput

2. Columnar storage
    - rows vs columns
    - column-oriented storage
    - compression
    - analytical scanning
3. ClickHouse architecture
    - databases
    - tables
    - storage engines
    - queries
4. Analytical queries
    - aggregation
    - filtering
    - grouping
    - large scans
5. Data ingestion
    - batch insertion
    - streaming ingestion awareness
    - insert performance
6. Compression
    - column compression
    - compression benefits
    - storage vs CPU trade-off
7. ClickHouse performance
    - vectorized execution
    - columnar processing

- parallel query execution
- large-scale aggregation

8. Distributed ClickHouse awareness
    - shards
    - replicas
    - distributed tables
    - distributed queries
9. ClickHouse vs PostgreSQL
    - OLAP vs OLTP
    - indexing differences
    - workload selection
    - analytical performance
10. Trading-system integration
    - asynchronous writes
    - decoupling matching engine from analytics
    - event/log storage
    - failure isolation

44. nginx

Preparedness target: WORKING

nginx is relatively easy to learn, but reverse proxies and load balancing are important system-design concepts.

Subskills

1. nginx fundamentals
    - web server
    - reverse proxy

    - configuration
    - workers
2. Reverse proxy
    - client
    - nginx
    - upstream server
    - request forwarding
3. Static file serving
    - static assets
    - file serving
    - caching basics
4. Load balancing
    - upstream servers
    - round robin
    - weighted routing
    - failover awareness
5. Proxy configuration
    - upstream blocks
    - proxy headers
    - proxy timeouts
    - request forwarding
6. Traffic routing
    - path-based routing
    - host-based routing
    - upstream selection
7. Traffic mirroring

- primary traffic
- shadow traffic
- duplicate requests
- non-production evaluation

8. nginx and application architecture
    - frontend
    - reverse proxy
    - backend
    - database
    - TLS termination awareness
9. nginx troubleshooting
    - logs
    - configuration errors
    - upstream failures
    - connection timeouts

---

45. OpenTelemetry

Preparedness target: WORKING

The important thing is understanding observability and how tracing helps diagnose distributed systems.

Subskills

1. Observability
    - logs
    - metrics
    - traces

- monitoring vs observability

2. Distributed tracing
    - request trace
    - spans
    - parent/child relationships
    - trace propagation
3. Spans
    - span lifecycle
    - attributes
    - events
    - status
    - duration
4. Trace context
    - trace IDs
    - span IDs
    - context propagation
    - correlation
5. Metrics
    - counters
    - gauges
    - histograms
    - latency metrics
6. Instrumentation
    - manual instrumentation
    - automatic instrumentation
    - application instrumentation

7. Service-to-service tracing
    - distributed requests
    - propagation
    - identifying slow dependencies
8. Observability architecture
    - application
    - collector
    - backend
    - visualization
9. Debugging with traces
    - latency bottlenecks
    - failed requests
    - dependency failures
    - request paths
10. Metrics vs logs vs traces
    - when to use each
    - complementary information
    - debugging workflows

---

46. Inverted Index

Preparedness target: VERY STRONG

This is one of your specialized strengths. Since your Search & Retrieval Engine is explicitly built around this, you should be able to explain the data structure from first principles.

Subskills

1. Information retrieval fundamentals

- documents
- terms
- queries
- relevance
- retrieval

2. Forward index vs inverted index
    - document → terms
    - term → documents
    - trade-offs
3. Dictionary
    - vocabulary
    - term IDs
    - term lookup
    - vocabulary storage
4. Postings lists
    - document IDs
    - postings
    - document frequency
    - sorted postings
5. Positional inverted index
    - term positions
    - phrase queries
    - proximity queries
6. Term frequency
    - occurrences
    - frequency counting

- document-level frequency

7. Document frequency
    - number of documents containing term
    - IDF relationship
8. Index construction
    - tokenization
    - normalization
    - term insertion
    - postings generation
9. Sorting postings
    - sorted document IDs
    - merge-based construction
    - efficient retrieval
10. Index compression
    - delta encoding
    - variable-length integers
    - compression benefits
11. LEB128 in inverted indexes
    - variable-length encoding
    - position compression
    - delta encoding
12. Query processing
    - term lookup
    - postings retrieval
    - postings intersection
    - multi-term queries

13. Boolean retrieval
    - AND
    - OR
    - NOT
    - postings intersection/union
14. Positional queries
    - phrase matching
    - proximity
    - positional intersection
15. Inverted-index complexity
    - indexing cost
    - query cost
    - memory requirements
    - corpus scaling
16. Index architecture
    - flat arrays
    - offsets
    - memory locality
    - cache-friendly access
17. Cross-language index formats
    - binary representation
    - versioning
    - byte-level compatibility
    - portability
18. Inverted-index engineering
    - memory constraints

- serialization
- loading
- corruption detection

---

47. BM25

Preparedness target: VERY STRONG

Because you implemented BM25 from scratch, this should be **explainable mathematically and intuitively**.

Subskills

1. Information retrieval scoring
   - relevance scoring
   - ranking
   - query-document matching
2. Term Frequency
   - TF
   - term occurrence
   - term saturation
3. Inverse Document Frequency
   - IDF
   - rare terms
   - common terms
   - corpus frequency
4. BM25 intuition
   - relevance
   - term importance

- document length
- term saturation

5. BM25 formula
    - TF component
    - IDF component
    - normalization
    - scoring
6. BM25 parameters
    - k1
    - b
    - parameter effects
7. Term saturation
    - diminishing returns
    - repeated terms
    - why raw TF is insufficient
8. Document-length normalization
    - short vs long documents
    - length normalization
    - b parameter
9. Multi-term BM25
    - scoring each query term
    - score aggregation
    - query processing
10. BM25 vs TF-IDF
    - normalization differences
    - saturation

    - ranking behavior
11. BM25 implementation
    - postings lists
    - document statistics
    - efficient scoring
12. BM25 optimization
    - candidate documents
    - postings traversal
    - memory locality
13. BM25 limitations
    - semantic mismatch
    - synonyms
    - vocabulary mismatch
    - contextual meaning
14. Hybrid retrieval
    - BM25
    - embeddings
    - PageRank
    - score fusion
15. BM25 evaluation
    - relevance judgments
    - ranking metrics
    - nDCG
    - precision@K

48. PageRank

Preparedness target: STRONG / INTERVIEW-READY

You have it in your search engine, so understand both the algorithm and why it can introduce bias.

Subskills

1. Graph-based ranking
    - nodes
    - edges
    - authority
    - centrality
2. Random-surfer model
    - random walk
    - surfer
    - transition probabilities
3. PageRank intuition
    - important pages linked by important pages
    - recursive authority
4. Transition matrix
    - graph representation
    - transition probabilities
    - stochastic matrix
5. Damping factor
    - teleportation
    - damping
    - avoiding traps
6. Power iteration
    - iterative computation

- convergence
- initialization

7. Dangling nodes
    - pages without outgoing links
    - handling dangling probability
8. Convergence
    - stopping criteria
    - tolerance
    - iteration count
9. PageRank complexity
    - graph size
    - sparse representation
    - iterative cost
10. PageRank in search
    - relevance signal
    - authority
    - combination with textual relevance
11. PageRank vs textual ranking
    - BM25
    - PageRank
    - semantic similarity
    - complementary signals
12. PageRank bias
    - popularity bias
    - selection bias
    - relevance judgment distortion

13. Weighted ranking
    - score normalization
    - weighted fusion
    - min-max normalization
14. PageRank limitations
    - topic mismatch
    - popularity vs relevance
    - graph manipulation
    - stale graph structure

---

49. TF-IDF

Preparedness target: WORKING → STRONG

Subskills

1. Bag-of-words representation
    - vocabulary
    - term counts
    - sparse vectors
2. Term Frequency
    - raw TF
    - normalized TF
    - term importance
3. Inverse Document Frequency
    - document frequency
    - rare vs common terms
    - logarithmic scaling

4. TF-IDF calculation
    - TF × IDF
    - vector construction
5. Sparse representation
    - sparse vectors
    - memory efficiency
6. Cosine similarity
    - vector angle
    - similarity score
    - normalization
7. TF-IDF document ranking
    - query vector
    - document vectors
    - similarity ranking
8. TF-IDF vs BM25
    - normalization
    - term saturation
    - document-length handling
9. TF-IDF limitations
    - no semantic understanding
    - synonyms
    - word order
    - contextual meaning

---

50. Sentence Transformers

Preparedness target: STRONG

This is one of your ML/IR crossover skills.

Subskills

1. Embeddings
    - vector representations
    - semantic information
    - dimensionality
2. Transformer fundamentals
    - tokens
    - attention
    - self-attention
    - contextual representations
3. Sentence embeddings
    - fixed-dimensional representation
    - sentence-level semantics
    - pooling
4. Sentence Transformer architecture
    - transformer encoder
    - pooling
    - embedding generation
5. Semantic similarity
    - cosine similarity
    - vector distance
    - semantic matching
6. Embedding generation pipeline

- text preprocessing
- tokenization
- model inference
- vector storage

7. Embedding dimensionality
    - 384-dimensional embeddings
    - dimensionality trade-offs
    - storage cost
8. Semantic retrieval
    - query embedding
    - document embeddings
    - nearest-neighbour retrieval
9. Hybrid search
    - semantic retrieval
    - BM25
    - score fusion
    - ranking
10. Embedding limitations
    - ambiguity
    - domain mismatch
    - semantic drift
    - hallucination not directly relevant to embeddings
11. Embedding evaluation
    - similarity benchmarks
    - retrieval metrics
    - nDCG

- recall@K

12. Computational considerations
    - inference cost
    - batch inference
    - vector storage
    - approximate search

51. LEB128

Preparedness target: RESUME-DEFENSIBLE

This is specialized. You should understand why you used it and how it saves space.

Subskills

1. Variable-length integer encoding
    - variable byte representation
    - compact integers
2. LEB128 encoding
    - 7-bit payload
    - continuation bit
    - byte representation
3. LEB128 decoding
    - reading bytes
    - continuation detection
    - reconstructing integer
4. Signed vs unsigned LEB128
    - unsigned representation
    - signed representation awareness

5. Delta encoding
    - storing differences
    - sorted integer sequences
    - compression
6. LEB128 + delta encoding
    - postings compression
    - positional index compression
7. Space/time trade-offs
    - reduced storage
    - decoding overhead
    - memory efficiency
8. Binary file formats
    - byte ordering awareness
    - versioning
    - serialization

---

52. Suffix Arrays

Preparedness target: VERY STRONG / SPECIALIZED

Because you actually implemented SA-IS, this is one of the areas where you should be able to defend an algorithmic implementation in detail.

Subskills

1. String suffixes
    - suffix definition
    - suffix ordering
    - lexicographic comparison

2. Suffix array
    - definition
    - sorted suffix positions
    - representation
3. Naive suffix-array construction
    - generating suffixes
    - sorting
    - complexity
4. Efficient construction
    - comparison-based methods
    - doubling techniques
    - complexity
5. SA-IS overview
    - induced sorting
    - linear-time construction
6. S-type and L-type suffixes
    - classification
    - suffix relationships
7. LMS positions
    - leftmost S-type
    - LMS substrings
    - recursive reduction
8. Induced sorting
    - bucket concepts
    - inducing L-type suffixes
    - inducing S-type suffixes

9. SA-IS recursion
    - reduced problem
    - naming LMS substrings
    - recursive suffix array
10. Suffix-array search
    - binary search
    - pattern matching
    - search complexity
11. Suffix array vs suffix tree
    - memory
    - implementation
    - search performance
12. Suffix array vs FM-index
    - underlying structures
    - compressed representation
    - search capabilities
13. Suffix arrays in genome search
    - sequence indexing
    - large text
    - repeated patterns
14. Complexity analysis
    - construction complexity
    - query complexity
    - memory complexity
15. Engineering large suffix arrays
    - memory constraints

- integer representations
- serialization
- loading

16. Testing suffix-array implementations
    - brute-force reference
    - randomized tests
    - correctness verification

---

53. Burrows-Wheeler Transform

Preparedness target: STRONG / SPECIALIZED

Subskills

1. BWT intuition
    - transform
    - rotations
    - reordered characters
2. BWT construction
    - cyclic rotations
    - sorted rotations
    - suffix-array relationship
3. BWT output
    - transformed string
    - last column
    - primary index
4. BWT inversion
    - first-last property

- LF mapping
- reconstructing original text

5. BWT and suffix arrays
    - relationship
    - suffix ordering
    - derivation
6. BWT and compression
    - character clustering
    - run-length encoding awareness
7. BWT and FM-index
    - backward search
    - compressed text indexing
8. Genome-search applications
    - DNA sequences
    - exact pattern matching
    - large text
9. Complexity and memory
    - construction
    - transformation
    - inversion

54. FM-Index

Preparedness target: VERY STRONG / SPECIALIZED

This is one of the deepest specialized skills on your resume.

Subskills

1. FM-index motivation
    - compressed full-text indexing
    - search without scanning
2. BWT foundation
    - BWT
    - suffix array relationship
    - transformed text
3. C array
    - character counts
    - first occurrence positions
4. Occurrence/rank function
    - rank queries
    - character counts up to position
5. LF mapping
    - last-to-first mapping
    - suffix movement
6. Backward search
    - pattern processing right-to-left
    - interval narrowing
7. FM-index query
    - initial interval
    - character extension
    - final suffix interval
8. Pattern occurrence recovery
    - locating suffix positions
    - locate operation awareness

9. Count vs locate
    - counting occurrences
    - retrieving positions
10. Space efficiency
    - compressed representation
    - rank structures
    - memory trade-offs
11. FM-index construction
    - suffix array
    - BWT
    - auxiliary structures
12. FM-index complexity
    - preprocessing
    - query complexity
    - memory complexity
13. FM-index vs suffix array
    - compression
    - query operations
    - implementation complexity
14. FM-index for genomic search
    - exact DNA matching
    - large genomes
    - scaling
15. Testing FM-index
    - brute-force comparison
    - randomized patterns

- correctness testing

55. Bloom Filters

Preparedness target: STRONG

Bloom filters are extremely common in systems interviews and are worth learning well.

Subskills

1. Probabilistic data structures
    - approximation
    - memory/time trade-offs
2. Bloom-filter structure
    - bit array
    - hash functions
    - membership query
3. Insert operation
    - hashing
    - setting bits
4. Lookup operation
    - hash computation
    - checking bits
5. False positives
    - why they occur
    - probability
6. False negatives
    - why standard Bloom filters have none

7. Number of hash functions
    - k
    - trade-offs
    - optimal k awareness
8. Bit-array sizing
    - memory requirement
    - false-positive rate
9. Bloom-filter tuning
    - expected elements
    - target false-positive probability
10. Bloom filters in distributed systems
    - cache penetration
    - storage engines
    - databases
    - network optimization
11. Bloom filter limitations
    - no deletion in standard version
    - false positives
    - capacity assumptions
12. Bloom filter variants
    - counting Bloom filters
    - scalable Bloom filters awareness
13. Bloom filters vs exact sets
    - memory
    - accuracy
    - use cases

56. MinHash

Preparedness target: WORKING / INTERVIEW-READY

Subskills

1. Set similarity
    - Jaccard similarity
    - intersection
    - union
2. MinHash intuition
    - randomized signatures
    - similarity approximation
3. Hash functions
    - permutations/hashes
    - minimum hash values
4. MinHash signature
    - multiple hash functions
    - signature vector
5. Jaccard estimation
    - matching signature components
    - approximate similarity
6. Error/accuracy trade-offs
    - number of hashes
    - signature size
    - approximation quality
7. MinHash applications

- near-duplicate detection
- document similarity
- sequence similarity

8. MinHash limitations
    - approximate result
    - parameter selection
    - sensitivity to representation
9. MinHash vs Bloom filters
    - similarity vs membership
    - probabilistic guarantees
    - use cases

57. scikit-learn

Preparedness target: STRONG

This should be practical rather than framework-trivia-heavy.

Subskills

1. Dataset preparation
    - loading data
    - train/test split
    - feature/label separation
2. Preprocessing
    - scaling
    - normalization
    - encoding
    - imputation

3. Pipelines
    - preprocessing pipeline
    - model pipeline
    - avoiding leakage
4. Classical algorithms
    - linear models
    - logistic regression
    - SVM
    - trees
    - random forests
5. Model selection
    - train/test
    - cross-validation
    - validation sets
6. Hyperparameter tuning
    - grid search
    - randomized search
    - parameter selection
7. Classification metrics
    - accuracy
    - precision
    - recall
    - F1
    - ROC-AUC
8. Regression metrics
    - MAE

- MSE
- RMSE
- R^2

9. Feature engineering
    - transformations
    - interactions
    - categorical features
    - feature selection
10. Data leakage
    - train/test contamination
    - preprocessing leakage
    - target leakage
11. Model persistence
    - saving models
    - loading models
    - reproducibility
12. Reproducibility
    - random seeds
    - deterministic experiments
    - experiment configuration
13. sklearn debugging
    - shape errors
    - preprocessing mismatch
    - feature mismatch
    - pipeline errors

---

58. PyTorch

Preparedness target: STRONG

You used PyTorch in the genomic project, so you should understand the training pipeline end-to-end.

Subskills

1. Tensor fundamentals
    - tensors
    - shapes
    - dimensions
    - dtypes
    - device
2. Tensor operations
    - indexing
    - broadcasting
    - matrix operations
    - reshaping
3. Autograd
    - computation graph
    - gradients
    - backward pass
    - gradient accumulation
4. Neural-network modules
    - nn.Module
    - layers
    - forward pass

5. Dataset and DataLoader
    - Dataset
    - DataLoader
    - batching
    - shuffling
    - workers
6. Training loop
    - forward pass
    - loss
    - backward
    - optimizer step
    - zeroing gradients
7. Loss functions
    - cross entropy
    - MSE
    - binary cross entropy
    - choosing losses
8. Optimizers
    - SGD
    - momentum
    - Adam
    - learning rate
9. Learning-rate management
    - learning rate
    - schedulers
    - warm-up awareness

10. Model evaluation
    - train/eval modes
    - validation
    - inference
    - metrics
11. Overfitting control
    - dropout
    - weight decay
    - early stopping
    - augmentation awareness
12. GPU training
    - CPU vs GPU
    - device placement
    - CUDA awareness
    - memory management
13. Model saving/loading
    - state_dict
    - checkpoints
    - resuming training
14. Reproducibility
    - random seeds
    - deterministic settings
    - experiment tracking awareness
15. PyTorch debugging
    - shape mismatches
    - NaNs

- exploding gradients
- vanishing gradients
- device mismatch

16. PyTorch performance
    - batching
    - data loading
    - GPU utilization
    - memory usage

---

59. Classical Machine Learning

Preparedness target: VERY STRONG

This is a **core ML interview domain**, so unlike individual libraries, it deserves significant depth.

Subskills

1. ML problem formulation
    - supervised learning
    - unsupervised learning
    - classification
    - regression
    - clustering
2. Training/validation/test split
    - training data
    - validation data
    - test data
    - cross-validation

3. Bias-variance trade-off
    - bias
    - variance
    - underfitting
    - overfitting
4. Linear regression
    - hypothesis
    - least squares
    - loss
    - coefficients
    - assumptions
5. Logistic regression
    - sigmoid
    - logits
    - binary classification
    - log loss
6. Regularization
    - L1
    - L2
    - sparsity
    - shrinkage
    - regularization strength
7. K-nearest neighbours
    - distance
    - neighbourhood
    - classification

- regression
- computational cost

8. Naive Bayes
    - Bayes theorem
    - conditional independence
    - likelihood
    - classification
9. Decision trees
    - splitting
    - entropy
    - information gain
    - Gini impurity
    - tree depth
10. Random forests
    - bagging
    - bootstrap samples
    - feature randomness
    - ensemble averaging
11. Gradient boosting
    - weak learners
    - sequential fitting
    - residual errors
    - boosting intuition
12. SVM
    - margin
    - support vectors

- maximum-margin classifier
- soft margin

13. Kernels
    - kernel trick
    - linear kernel
    - polynomial kernel
    - RBF kernel
14. Feature engineering
    - scaling
    - transformations
    - categorical encoding
    - feature selection
15. Missing data
    - missingness
    - imputation
    - missing indicators
16. Class imbalance
    - imbalance
    - class weights
    - resampling
    - precision/recall trade-off
17. Model evaluation
    - confusion matrix
    - precision
    - recall
    - F1

- ROC-AUC
- PR-AUC

18. Calibration
    - probability estimates
    - calibration
    - reliability
19. Hyperparameter tuning
    - grid search
    - random search
    - validation
    - overfitting to validation
20. Cross-validation
    - K-fold
    - stratified K-fold
    - time-series split awareness
21. Data leakage
    - target leakage
    - preprocessing leakage
    - train/test contamination
22. Feature importance
    - tree importance
    - permutation importance
    - model coefficients
23. Model interpretability
    - feature importance
    - local explanations

- global explanations

24. Statistical foundations
    - distributions
    - expectation
    - variance
    - covariance
    - correlation
25. Generalization
    - empirical risk
    - generalization error
    - unseen data
26. Experimental methodology
    - controlled experiments
    - baselines
    - random seeds
    - repeated runs
    - statistical comparison
27. Classical ML trade-offs
    - accuracy vs interpretability
    - complexity vs performance
    - training vs inference cost
    - bias vs variance
28. Choosing models
    - dataset size
    - feature types
    - interpretability

- computational constraints
- linear vs nonlinear models